

INTERNSHIP REPORT

User Management System with Secure Authentication

WEEK 1 • 2026

INTERN NAME

Jil Patel

UNIVERSITY

Pandit Deendayal Energy University

DURATION

18 May 2026 – 24 May 2026

REPORTING MANAGER

Hemal Shah (HR)

INTERNSHIP ROLE

SoftwareDeveloper Intern

WORK MODE

On-Site

COMPANY

ncode Solutions — A Division of GNFC Ltd.

TECH STACK

ASP.NET Core MVC (C#) • PostgreSQL 16 • Entity Framework Core 8

Bootstrap 5 • ASP.NET Core Identity • JWT (RFC 7519) • Postman

SECTION 1

Project Overview & Objectives

Intern Name	Jil Patel
Roll Number	24BCP098
University	Pandit Deendayal Energy University
Project Title	User Management System with Secure Authentication
Company	ncode Solutions — A Division of GNFC Ltd.
Company Address	(n) Code Solutions, 14th Floor, Tower One, Road 5C, Zone 5, GIFT City, Gandhinagar, Gujarat 382050
Internship Role	Software Developer Intern
Reporting Manager	Hemal Shah (HR)
Work Mode	On-Site
Duration	18 May 2026 – 24 May 2026

The core objective during the first week of my internship at ncode Solutions (GNFC) was to initiate, architect, and deliver a production-ready User Management System (UMS). This system serves as an enterprise foundation for secure identity management, requiring a hybrid authentication architecture — stateful browser sessions for the standard web UI and stateless, cryptographically signed tokens to secure modular API endpoints.

The UMS was designed to handle real-world enterprise requirements including role-based access control, encrypted credential storage, server-side input validation, and audit-ready data structures. The project demanded a clean separation of concerns across MVC layers, ensuring maintainability, testability, and scalability with growing organizational requirements.

1.1 Architecture Objectives

- Build a scalable MVC folder architecture decoupling business logic from data-access layers.
- Model a secure relational database structure mapping users, roles, and privileges using PostgreSQL.
- Enforce strict server-side model validations and integrity constraints to minimize security risk vectors.
- Deploy robust identity protection features including cryptographic password hashing via ASP.NET Core Identity.
- Implement token-gated API endpoint access using JSON Web Tokens (JWT) following industry-standard practices.
- Create a responsive, professional-grade web interface using Bootstrap 5 with dynamic role-aware UI rendering.

1.2 Business & Functional Requirements

Module	Requirement	Priority
User Registration	Capture and persist user data with validations and hashed passwords	High
Authentication	Verify credentials and establish secure session or token context	High
Role Management	Differentiate Admin and User privilege levels throughout the system	High

Dashboard Analytics	Aggregate and display system-wide user statistics for admins	Medium
CRUD Operations	Allow admins to view, edit, and delete user records	Medium
API Layer	Expose registration and authentication via RESTful JSON endpoints	Medium
JWT Token Auth	Protect API routes with signed Bearer tokens and [Authorize] filters	High

SECTION 2

Technologies Used

Category	Technology / Tool	Purpose
Backend Framework	ASP.NET Core MVC (C#)	Core application framework; request lifecycle, routing, middleware
Database	PostgreSQL 16	Relational data storage for users, roles and audit records
ORM	Entity Framework Core 8	Object-relational mapping; migrations, LINQ queries, DbContext
Frontend	Bootstrap 5 + HTML5 + CSS3	Responsive UI layout, components, styling, and adaptive grids
Authentication	ASP.NET Core Identity + JWT Bearer	Password hashing, session management, and stateless token auth
Token Standard	JSON Web Tokens (JWT / RFC 7519)	Cryptographically signed tokens for API endpoint protection
Development IDE	Visual Studio 2022	Full project scaffolding, NuGet management, debugging
Database GUI	pgAdmin 4	Visual schema inspection, query execution, table management
API Testing	Postman	HTTP request simulation, response validation, auth header testing
Package Manager	NuGet	Dependency resolution and library installation

SECTION 3

Detailed Day-wise Work Description

Day 1 • 18 May 2026

Project Setup, Dependency Injection & Core Middleware Configuration

Initialized the baseline web application using Visual Studio 2022 scaffolding templates. Built a strict Model-View- Controller (MVC) directory structure to decouple data objects, processing operations, and client-facing layouts, establishing clear separation of concerns from the very first commit.

- Configured the native Dependency Injection (DI) container within **Program.cs** to manage service lifecycles using the built-in IoC container.
- Installed core data management drivers via NuGet: **Microsoft.EntityFrameworkCore**, **Microsoft.EntityFrameworkCore.Design**, and **Npgsql.EntityFrameworkCore.PostgreSQL**.
- Formulated global configurations inside **appsettings.json**, declaring physical database connection strings to establish a runtime pipeline with the local PostgreSQL environment.
- Registered essential middleware in the correct pipeline order: routing, static files, authentication, authorization, and session handling.

Day 2 • 19 May 2026

Relational Database Modeling, Schema Mapping & EF Core Migrations

Designed the persistent storage layout by crafting specialized entity representations using strongly typed C# classes. This step formed the entire data foundation for the project.

- Authored a foundational **UserMaster** entity model tracking: FullName, Username, Email, MobileNumber, Password (hashed), DateOfBirth, RoleId, and CreatedDate.
- Designed a separate **RoleMaster** table (storing Admin and User roles) linked via foreign key constraints to UserMaster — enforcing role boundaries at the schema layer.
- Declared an **AppDbContext** class inheriting from DbContext, mapping entity collections using **DbSet<UserMaster>** and **DbSet<RoleMaster>**.
- Executed database migrations via Package Manager Console: **Add-Migration InitialCreate** followed by **Update-Database**, converting abstract C# definitions into physical PostgreSQL tables.

Day 3 • 20 May 2026

Secured Registration Workflow & Server-Side Input Auditing

Engineered a full user onboarding system capable of screening input data integrity and blocking invalid entries prior to database ingestion — the first line of defense against bad data and security vulnerabilities.

- Constructed declarative validation schemes using DataAnnotations attributes: **[Required]**, **[EmailAddress]**, **[StringLength]**, and **[RegularExpression]** on registration view models.
- Implemented server-side logic enforcing strict confirmation string parity — matching Password and Confirm Password fields with descriptive error messaging.
- Enforced business constraints explicitly rejecting registrations submitting a future date as the user's Date of Birth.
- Mitigated data-duplication risks by deploying asynchronous LINQ uniqueness checks using **.AnyAsync()** to verify Email and Mobile Number uniqueness before insert.
- Prevented plain-text credential storage by routing all password ingestion through ASP.NET Core's cryptographically secure **PasswordHasher<UserMaster>** engine.

Day 4 • 21 May 2026

State Maintenance, Cryptographic Verification & Session Lifecycles

Engineered complete login handling logic to authenticate credentials against database records and establish persistent web state mechanisms across user navigation.

- Formulated secure lookup queries verifying submitted usernames against database records; if found, the stored hash was evaluated using the **PasswordVerificationResult** utility.
- Activated cross-request state tracking by registering and configuring session middleware in Program.cs using **builder.Services.AddSession()** with configurable idle timeouts.
- Upon successful validation, cached user context into server-side memory using **HttpContext.Session.SetString()** and **SetInt32()**, capturing Username, UserId, and RoleId.
- Implemented an automated routing interceptor that checks session variables and redirects authenticated users to their role-appropriate landing dashboards.

Day 5 • 22 May 2026

Multi-Tier Metrics Dashboard & Privilege-Bound CRUD Controls

Constructed the administrative management interface with a real-time metrics dashboard and implemented rigorous role-based access restrictions across all data channels.

- Programmed a metrics dashboard backend executing optimized aggregate LINQ queries to compile real-time counters: Total System Users, Total Admin accounts, and Regular User distributions.
- Built a secure user profile view pulling identifiers directly from active session parameters to safely render only the session-owner's personal data.
- Locked down administrative CRUD controllers: regular users are systematically blocked via session role-checks, while validated admins can view full user lists, edit account properties, and trigger soft-deletion routines.
- Ensured unauthorized direct URL access is intercepted and redirected to the login screen, preventing privilege escalation via URL manipulation.

Day 6 • 24 May 2026

Responsive Layout Refactoring, Component Styling & Feedback Loops

Upgraded all front-end layouts into a polished, adaptive corporate interface using Bootstrap 5 structural components — significantly improving usability and visual consistency.

- Implemented a dual-axis dashboard layout with a collapsible sidebar navigation module and a top contextual navbar dynamically displaying the active session user's name and role badge.
- Formatted raw data lists into sortable, responsive data tables with color-coded role badges (Admin / User) for rapid visual scanning.
- Applied consistent card-based layouts for dashboard metric counters with icon support and hover animations.
- Deployed a professional feedback loop using **TempData** to enable the layout master to automatically surface animated Bootstrap alert banners on success, warning, or error events.

Day 7 • 25 May 2026

Stateless JWT Core Architecture & Postman API Evaluation

Expanded the system capabilities by architecting a full programmatic REST API layer running in parallel with traditional web view controllers — enabling the system to serve both browser clients and external integrations.

- Programmed a secure token assembly routine generating signed JSON Web Tokens (JWT) using HMAC-SHA256 — bundling Username, UserId, and RoleId claims inside the encrypted payload.
- Registered JWT Bearer authentication scheme in Program.cs, configuring issuer, audience, and signing key validation parameters via **TokenValidationParameters**.
- Created dedicated **ApiAuthController** and **ApiUserController** providing standardized JSON responses for registration and authentication transactions.
- Protected sensitive data endpoints using the **[Authorize]** attribute filter, requiring callers to present a verified, non-expired Bearer token in the HTTP Authorization header.
- Conducted comprehensive endpoint audits via Postman: verified 200 OK on valid credentials, 401 Unauthorized on missing tokens, and 403 Forbidden on insufficient role privileges across all user profiles.

SECTION 4

TaskStatus Summary

All 16 tasks planned for Week 1 were completed successfully. Each task is categorized by functional domain and marked with its completion status as evaluated at the end of the internship week.

#	Task Description	Category	Status
1	MVC Architectural Directory Setup & Dependency Mapping	Backend	✓ COMPLETED
2	PostgreSQL Provider Setup & Connection String Configuration	Database	✓ COMPLETED
3	DB Data Context Blueprinting & Data Table Relationships	Database	✓ COMPLETED
4	Entity Property Modeling & Database Schema Migrations	Database	✓ COMPLETED
5	Data Annotation Constraints & Server-Side Input Validation	Backend	✓ COMPLETED
6	PasswordHasher Cryptographic Integration	Security	✓ COMPLETED
7	Credential Auditing Logic & Onboarding Pipelines	Backend	✓ COMPLETED
8	Session Middleware Pipeline Activation (Program.cs)	Security	✓ COMPLETED
9	Stateful Property Ingestion (HttpContext.Session)	Security	✓ COMPLETED
10	Metrics Aggregator Queries & Analytics Dashboard Counters	Backend	✓ COMPLETED
11	Administrative Controller Boundaries & Action Filters	Security	✓ COMPLETED
12	Core UI Components Layout (Sidebar, Profile View, Tables)	Frontend	✓ COMPLETED
13	Bootstrap 5 Layout Refactoring & TempData Alert Banners	UI/UX	✓ COMPLETED
14	Symmetric Security Key Generation & JWT Claims Factory	Security	✓ COMPLETED
15	Token-Gated API Controllers with [Authorize] Filter	Security	✓ COMPLETED
16	End-to-End Route Validation & Payload Testing via Postman	Testing	✓ COMPLETED

SECTION 5

Challenges Faced & Solutions Implemented

Challenge 1: Dual-State Hybrid Authentication Conflicts
Problem Statement: Coordinating stateful cookie-based session tracking alongside stateless JWT Authorization headers caused internal middleware parsing conflicts. Web view controllers unintentionally required Bearer token headers, while API controllers were redirected to cookie-based login pages — producing cross-routing loops and incorrect 401/302 responses.
Solution Applied: Explicitly separated the two authentication configuration layers in Program.cs. Defined independent scheme routers: localized Cookie-based authentication strictly for MVC UI Controllers, and an independent JWT Bearer parsing engine bound exclusively to API-prefixed route controllers. Each controller type now carries explicit [Authorize(AuthenticationSchemes=...)] annotations to prevent scheme bleed-over.
Challenge 2: Time-Zone Type Faults with PostgreSQL DateTime Schema
Problem Statement: PostgreSQL rejected incoming database insert and update operations with InvalidCastException errors when attempting to persist native C# DateTime instances. The root cause was implicit time offset formatting: C# DateTime.Now carries local timezone offsets, which PostgreSQL's timestamptz columns rejected without explicit UTC normalization.
Solution Applied: Normalized all model-tier timestamp parameters. Configured application logic to systematically convert all DateTime values to UTC using DateTime.UtcNow before any EF Core tracking mutations or explicit database update commands. Added a global DateTimeKind enforcement comment in the DbContext for future developer reference.

Challenge 3: Route Binding Collisions and HTTP Error Code Traps
Problem Statement: Overlapping endpoint route definitions and missing route constraints caused frequent 400 Bad Request, 404 Not Found, and 405 Method Not Allowed responses during early Postman API testing. Conventional MVC action routes clashed with API controller routes sharing similar path segments.
Solution Applied: Refactored all controller routing configurations by deploying explicit routing tokens ([Route("api/[controller]")] and mapping distinct HTTP verb decorators ([HttpPost], [HttpGet], [HttpPut], [HttpDelete]) over all endpoint methods. Tested each endpoint in isolation through Postman before integration to confirm correct verb binding and expected HTTP status code responses.

SECTION 6

Key Learning Outcomes

Week 1 provided extensive hands-on exposure to enterprise software development practices. The following learning outcomes were achieved across technical, architectural, and professional dimensions.

1. Enterprise Framework Architecture

Gained deep practical understanding of the ASP.NET Core MVC request lifecycle — from HTTP request entry through middleware pipelines, controller action resolution, model binding, validation, and view rendering. Learned how dependency injection containers manage service lifetimes (Singleton, Scoped, Transient) and why choosing the correct lifetime prevents subtle concurrency bugs.

2. Relational Database Management

Mastered the full workflow of Entity Framework Core with PostgreSQL: designing normalized relational schemas, applying DataAnnotations and Fluent API constraints, running incremental migrations, and writing LINQ queries that translate efficiently to SQL. Understood the importance of foreign key integrity and UTC datetime normalization when working with cross-timezone enterprise databases.

3. Hybrid Security Architecture

Acquired technical proficiency in implementing both stateful server-side Sessions (cookie-based, appropriate for web UI contexts) and stateless cryptographic JWT tokens (appropriate for API contexts). Understood when each pattern is suitable and how to configure them without conflict in the same application — a critical real-world production requirement.

4. Production Diagnostics & API Testing

Gained proficiency in analyzing HTTP request/response structures using Postman: setting Authorization headers, simulating different user roles, inspecting status codes, and tracing error responses back to specific route or validation failures. Developed systematic debugging habits by isolating one variable at a time and reading framework-level exception stack traces carefully.

5. UI/UX Engineering Awareness

Learned how a well-structured Bootstrap 5 grid system, semantic HTML components, and contextual feedback patterns (TempData alerts, role badges) collectively improve usability for end users. Understood the importance of responsive design and how CSS media queries enable the same interface to serve both desktop and mobile contexts without duplication.

6. Professional Software Development Practices

Internalized the importance of modular code organization, consistent naming conventions, and separating configuration from code (appsettings.json). Learned how to scope a week's worth of work into achievable daily milestones and document decisions clearly for team transparency.

SECTION 7

Conclusion & Future Scope

7.1 Conclusion

The first week of my internship at ncode Solutions (GNFC) successfully achieved all its structural and functional objectives. Over seven intensive working days, I designed and delivered a fully operational, high-security User Management System from scratch — covering relational database schema design, backend authentication logic, responsive frontend layout, and a JWT-secured REST API layer.

Every component was built to production standards: passwords are never stored in plain text, API routes are protected behind verified tokens, and UI access is gated by server-side role checks rather than client-side conditionals alone. The project provided genuine full-stack exposure that closely mirrors real-world enterprise software development workflows.

The technical challenges encountered — particularly the hybrid authentication configuration and PostgreSQL datetime handling — pushed me to read official documentation, inspect framework source behavior, and reason through root causes systematically rather than relying on trial-and-error. This diagnostic approach will remain a core professional habit beyond this internship.

7.2 Future Scope — Planned Enhancements for Upcoming Weeks

Feature	Description	Target
Email Verification	Send OTP/link to registered email to confirm account ownership before activation	Week 2
Forgot Password	Secure password reset flow via time-limited email token with SMTP integration	Week 2
Two-Factor Authentication	Add TOTP-based 2FA (Google Authenticator compatible) as an optional layer for admins	Week 3
Audit Logging	Record login events, data mutations, and admin actions to a timestamped audit trail table	Week 3
Soft Delete & Archive	Replace hard deletes with soft-delete flags and user archive views for data recovery	Week 3
Cloud Deployment	Deploy the complete application to a cloud environment (Azure App Service or AWS EC2)	Week 4
Refresh Token Mechanism	Implement JWT refresh tokens to extend sessions without re-authentication	Week 4